



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第2讲 Python语言基础



Python课程组



2025年02月



CONTENTS

目 录

- | | | | |
|---|-------------|---|--------|
| 1 | 程序设计与计算问题 | 5 | 数字类型 |
| 2 | Python程序的构成 | 6 | 表达式与运算 |
| 3 | 编码规范 | 7 | 字符串类型 |
| 4 | 对象与变量 | 8 | 实践指导 |



武汉大学
WUHAN UNIVERSITY

01

程序设计与计算问题



引例

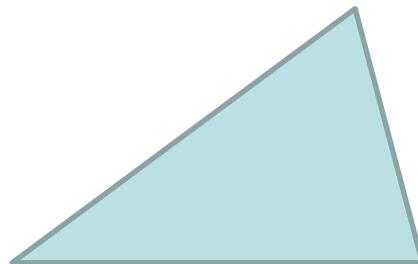


武汉大学
WUHAN UNIVERSITY

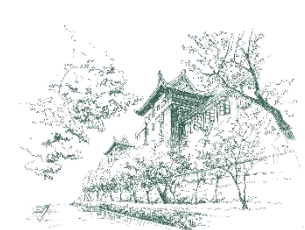
■ 问题：计算三角形的面积

- ✱ 三角形形状任意

- ✱ 三角形大小任意



■ 那么，如何编写程序，让计算机解决这个计算问题？



程序设计与计算问题



武汉大学
WUHAN UNIVERSITY

■ 程序设计的核心目标是解决计算问题。

■ 什么是计算问题？

✱ 计算问题是指那些可以通过明确的步骤和规则，利用计算机进行处理的问题。

✱ 计算问题通常具有以下特点：

- 明确的输入和输出： 问题有清晰的输入数据，并且需要得到特定的输出结果。
- 可分解性： 问题可以被分解成更小的子问题，每个子问题都可以独立解决。
- 可计算性： 问题可以通过算法和程序来解决。

✱ 例子：

- 计算两个数的和。
- 查找一组数据中的最大值。
- 对一组数据进行排序。
- 预测天气（基于历史数据和模型）。



解决计算问题的步骤



武汉大学
WUHAN UNIVERSITY

■ 程序设计通过以下步骤解决计算问题：

✿ 问题分析

- 明确问题的输入、输出和约束条件。

✿ 算法设计

- 设计解决问题的步骤和方法。

✿ 编写代码

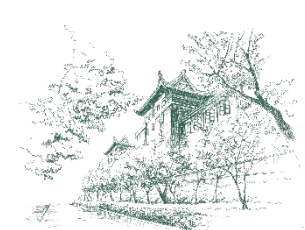
- 使用编程语言将算法转化为计算机可以执行的代码。

✿ 调试和测试

- 运行程序，检查是否能够正确解决问题。

✿ 优化和维护

- 优化代码的性能和可读性，并根据需求变化进行维护。



计算问题的分类



■ 计算问题可以根据其复杂性和解决方法进行分类：

✿ 简单计算问题

- 问题规模小，解决方法直观。
- 例如：计算两个数的和、判断一个数是否为素数。

✿ 复杂计算问题

- 问题规模大，解决方法需要更复杂的算法和数据结构。
- 例如：排序大量数据、查找最短路径、处理三维图像。

✿ 开放性问题

- 问题没有明确的解决方法，需要创新和探索。
- 例如：人工智能、自然语言处理、量子计算。



程序设计与计算问题密不可分



武汉大学
WUHAN UNIVERSITY

■ 程序设计是解决计算问题的工具

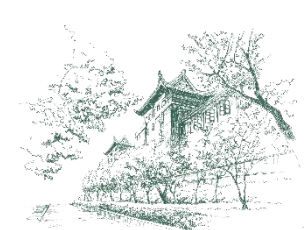
✿ 通过编写程序，我们可以将计算问题的解决方法自动化。

■ 计算问题是程序设计的驱动力

✿ 计算问题的复杂性和多样性推动了程序设计方法和工具的发展。

■ 程序设计的核心是算法

✿ 算法是解决计算问题的关键，程序设计则是将算法转化为可执行代码的过程。



用IPO方法编写程序解决计算问题

- IPO方法 (Input-Process-Output) 是一种简单而有效的程序设计思路，特别适合初学者理解和解决计算问题。
- 它将程序分为三个部分：输入、处理和输出。
- IPO方法的优势
 - ✿ 简单直观： 将问题分解为三个明确的步骤，易于理解和实现。
 - ✿ 结构化： 帮助初学者养成良好的编程习惯，避免代码混乱。
 - ✿ 通用性强： 适用于大多数简单的计算问题。



IPO方法的基本步骤



■ 输入：明确程序需要哪些数据或信息作为输入。

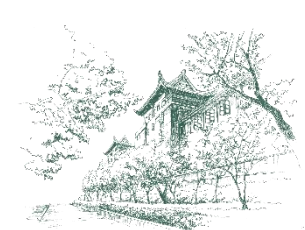
- ✿ 输入数据类型：确定输入数据的类型，如数字、文本、文件等。
- ✿ 输入数据来源：数据可能来自用户输入、文件、数据库或其他程序。

■ 处理：设计算法来处理输入的数据。

- ✿ 算法是解决问题的一系列步骤，是一个程序的灵魂。
- ✿ 逻辑结构：使用控制结构（如分支、循环）来组织算法的逻辑。
- ✿ 数据结构：选择合适的数据结构来存储和处理数据。

■ 输出：明确程序需要产生哪些结果或输出。

- ✿ 输出格式：确定输出的格式，如文本、图表、报告等。
- ✿ 输出目的地：决定输出的目标位置，如屏幕、文件、打印机等。



引例：设计三角形面积计算程序

■ 需求分析

- ✱ 明确目标：编写一个程序，每次运行能计算一个已知三边长度的三角形的面积。

■ 设计（用IPO方法）

✱ 输入

- 每次执行三角形面积计算程序，首先需要用户敲击键盘输入三角形的三边长度，也就是三个数（整数或者小数）。

✱ 处理

- 基于三边长度计算三角形面积的常用方法是海伦公式。
- 设三条边长分别为 a 、 b 和 c ，则三角形面积 s 的计算公式为：
$$s = \sqrt{h(h-a)(h-b)(h-c)}$$
- 其中， h 为三角形周长的一半。

✱ 输出

- 把计算出的面积值（一个数），显示在屏幕上。



引例：编写三角形面积计算程序



■ 编写三角形面积计算程序的代码。

✿ 源文件：triangle_area.py

```
import math

print("输入三角形的三边长度：")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

h = (a + b + c) / 2
area = math.sqrt(h * (h - a) * (h - b) * (h - c))

print("三角形的面积为：", area)
```

✿ 执行程序

```
输入三角形的三边长度：
a: 3
b: 4
c: 5
三角形的面积为： 6.0
```



武汉大学
WUHAN UNIVERSITY

02

Python程序的构成



引例



武汉大学
WUHAN UNIVERSITY

■ 三角形面积计算程序的源代码（源文件为triangle_area.py）

语句是一个独立且完整的语法单位。执行程序，就是逐条执行程序中的语句。

语句

空行

为了把空行上下两部分语句隔开，增强代码的可读性。

```
triangle_area.py - C:\Users\zhanghua\Desktop\mypython\p02\triangle_area.py (3.11.4)
File Edit Format Run Options Window Help
1 import math
2
3 print("输入三角形的三边长度：")
4 a = float(input("a: "))
5 b = float(input("b: "))
6 c = float(input("c: "))
7
8 h = (a + b + c)/2
9 area = math.sqrt(h * (h - a) * (h - b) * (h - c))
10
11 print("三角形的面积为：", area)
```

Ln: 11 Col: 0



Python程序的构成



武汉大学
WUHAN UNIVERSITY

Python的语句

语句是Python程序的过程构造块，用于定义函数、定义类、创建对象、变量赋值、调用函数、控制分支、创建循环等。

Python语句分为简单语句和复合语句。

简单语句包括：

➤ 表达式语句、赋值语句、**assert**语句、**pass**空语句、**del**语句、**return**语句、**yield**语句、**raise**语句、**break**语句、**continue**语句、**import**语句、**global**语句、**nonlocal**语句等。

复合语句包括：

➤ **if**语句、**while**语句、**for**语句、**try**语句、**with**语句、函数定义**def**、类定义**class**等。

```
import math
```

```
print("输入三角形的三边长度：")
```

```
a = float(input("a: "))
```

```
b = float(input("b: "))
```

```
c = float(input("c: "))
```

```
h = (a + b + c) / 2
```

```
area = math.sqrt(h * (h - a) * (h - b) * (h - c))
```

```
print("三角形的面积为：", area)
```

简单语句

```
''' 输入3个整数，输出最大数 '''
```

```
a, b, c = eval(input(' 输入3个整数，用逗号隔开: '))
```

```
if a >= b:
```

```
    if a >= c:
```

```
        print(f"最大数是{a}")
```

```
    else:
```

```
        print(f"最大数是{c}")
```

```
else:
```

```
    if b >= c:
```

```
        print(f"最大数是{b}")
```

```
    else:
```

```
        print(f"最大数是{c}")
```

复合语句



Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 导入语句

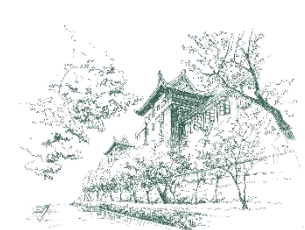
- ✿ `import math`

- ✿ 导入模块`math`。

- ✿ 模块通常包含用户自定义的变量、函数和类。

- ✿ `math`是Python标准库中的一个模块。

- 定义了一些实现算术运算的函数，例如这个程序要用到计算平方根的函数`sqrt`。
- 还包含三角函数`sin`、对数函数`log`等。



Python程序的构成

■ 模块

- ✿ 一个Python程序由1个或多个模块构成，每个模块是一个源文件（.py），例如 `triangle_area.py`。
- ✿ 一个模块可以被其他模块导入执行，也可以独立执行。

■ 包

- ✿ 把一个程序的若干模块文件放在一个文件夹中，称为“打包”。
- ✿ 包是模块的层次性组织结构，功能相近的模块可以组织成包。

■ 库

- ✿ 就是定义了函数、常量、数据类型等的若干程序文件的集合，可以被应用程序调用去完成特定任务。经常与模块和包混用。
- ✿ 可以简单地认为：模块、包和库预置了一组可供使用的功能。



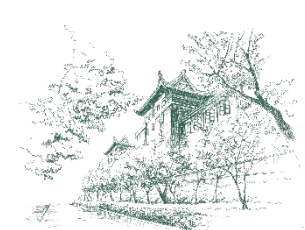
Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 标准库

- ✿ 提前编写好的程序文件，随着安装Python一起被安装在编程环境中。
- ✿ Python的特点之一是它的标准库中有大量的实用模块。



Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 内置函数print

- ✿ 以文本形式输出数据。
- ✿ 定义在Python的内置模块中。

■ 内置模块

- ✿ 名称为：__builtins__
- ✿ Python程序不需要用import语句导入内置模块，而是可以直接引用内置模块中定义的函数和数据类型。



内置对象与非内置对象

■ 内置对象

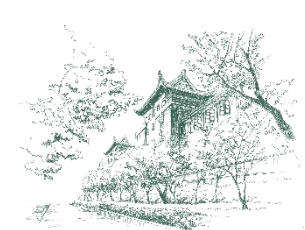
✿ Python内置模块中定义的函数、数据类型、异常等，被称为内置对象。

✿ 例如

- 内置数据类型：int、float、complex、str、list、dict等
- 内置函数：print、input、abs、pow、max、sum、eval等

✿ 用语句 `dir(__builtins__)` 查看所有内置对象

```
Anaconda Prompt - python
(base) C:\Users\zhanghua>python
Python 3.11.4 | packaged by Anaconda | (main, Jul 5 2023, 13:38:37) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits()" or "license()" for more information.
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup', 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError', 'Exception', 'ExceptionGroup', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError', 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '__build_class__', '__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'aiter', 'all', 'anext', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes', 'callable', 'chr', 'class', 'method', 'compile', 'complex', 'copyright', 'credits', 'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit', 'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
>>>
```



内置对象与非内置对象



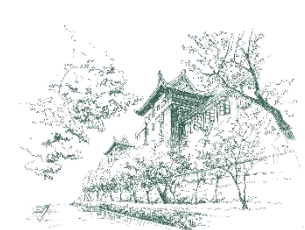
武汉大学
WUHAN UNIVERSITY

■ 非内置对象

✱ 需要导入相应的模块或模块的元素才能使用。

✱ 例如

- 使用标准库`math`模块中的平方根函数`sqrt`前，要用`import`导入`math`
- `import math`



函数



■ 函数是完成某项任务的一段程序，可以通过函数名来调用，即执行该段程序。

■ Python程序可以使用内置函数、标准库函数、扩展库函数。

✿ 内置函数

➤ 例如print、input、type、id、dir、help、len等。

✿ 标准库函数

➤ 例如标准库中模块math的函数sqrt。

✿ 扩展库函数

➤ 例如扩展库Numpy中的函数arange。

```
import math
print("输入三角形的三边长度：")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

h = (a + b + c) / 2
area = math.sqrt(h * (h - a) * (h - b) * (h - c))

print("三角形的面积为：", area)
```



调用模块函数



■ 怎么使用非内置函数

- ✱ 通过import语句，可以导入模块module，然后使用下面的形式调用模块中的函数。

模块名.函数名(参数)

- ✱ 一般，建议每个import语句只导入一个模块，如果从同一个模块中导入多个函数或类，可以使用逗号分隔的方式。

```
import csv
import datetime
from math import sqrt, cos, sin
import pandas as pd
import matplotlib.pyplot as plt
```

```
datetime.datetime.today()
```



Python程序的构成



■ 内置函数input

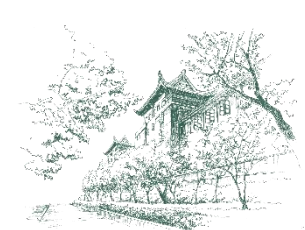
- ✿ 读取用户输入的数据。
- ✿ 但是，要注意的是，该函数返回的是字符串对象。

```
>>> a = input("a: ")  
a: 3  
>>> a  
'3'
```

■ 调用函数float把字符串对象转换为数字

- ✿ float是Python内置的数据类型之一：浮点数类型。
- ✿ float也是Python内置的函数，可以把字符串对象转换为数字。

```
>>> float('3')  
3.0
```



Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 变量和赋值

- ✱ 通过赋值运算 “=” 把创建的浮点数赋值给了三个变量a、b、c。
- ✱ 在Python程序中，变量是对象的标签，用来引用对象。
- ✱ 在形式上，变量是一个用户定义的名字（例如a、b、c、h、area），这个名字的正式称呼是标识符。



Python程序的构成

■ 标识符是由编程者自定义的用于给变量、函数、类、模块或其他对象命名的字符序列。

■ 标识符命名规则

- ✱ 只能使用下划线（_）、数字字符（0~9）、大写字母字符（A~Z）和小写字母字符（a~z）。
- ✱ 不能用数字字符开头。
- ✱ 标识符长度任意，区分大小写，不能使用关键字。

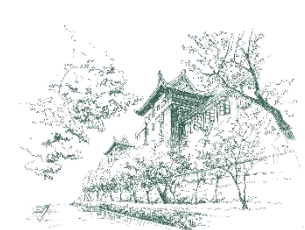
an_int、edgel、_strName、
BusNo、锅炉温度 ✓

99var、It'sOK、import ✗

```
import math
print("输入三角形的三边长度：")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

h = (a + b + c) / 2
area = math.sqrt(h * (h - a) * (h - b) * (h - c))

print("三角形的面积为：", area)
```

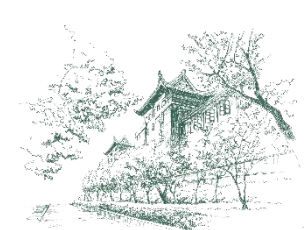
Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 关于标识符命名规则的特别说明

- ✿ 虽然Python的标识符中可以使用Unicode字符集中非ASCII字符的部分字符，例如中文字符。
- ✿ 但是，在编写跨平台的程序时，使用这些字符可能会引发编码问题。



Python程序的构成



标识符命名的注意事项

- ✱ 以下划线开头的变量在Python中有特殊含义。
- ✱ 以双下划线开始和结束的名称通常具有特殊的含义。
 - 例如，`__init__`为类的构造函数，一般应避免使用。
- ✱ 不建议使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义。
 - 例如，`int=12`，那么`int`就不再是内置的整数类型名了，导致混乱。
- ✱ 标识符作为变量、函数或类型名称时，最好包含完整的单词或者拼音，能反映所引用的量、函数的功能、类型的含义，方便阅读。



Python程序的构成

■ 关键字是预定义的名字，也称保留字，具有特定的含义和用途。

✿ Python 3有35个关键字

```
>>> help("keywords")
```

```
Here is a list of the Python keywords.  Enter any keyword to get more help.
```

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

✿ 某些标识符仅在特定上下文中被保留，被称为**软关键字**。Python 3.10增加了**match**、**case**和**_**，它们在**match**语句中使用。Python 3.12增加了**type**，在**type**语句中使用。



Python程序的构成



字面值

- ✱ 直接写出的数字（例如2、123、4.56、0.10）和字符串（例如"a: "、"三角形的面积为："、'李华'），被称为字面值。
- ✱ 字面值是某种类型常量值的表示方法。
- ✱ 执行程序时，每个字面值就是一个对象。
 - 例如，123是一个整数对象，'李华'是一个字符串对象。

算术运算表达式

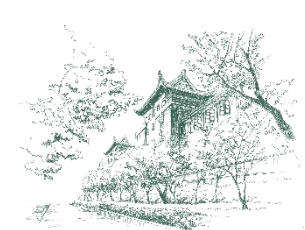
- ✱ 执行算术运算。

```
import math

print("输入三角形的三边长度：")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

h = (a + b + c) / 2
area = math.sqrt(h * (h - a) * (h - b) * (h - c))

print("三角形的面积为：", area)
```



Python程序的构成



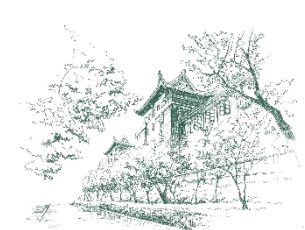
武汉大学
WUHAN UNIVERSITY

■ 注释用来对代码进行说明。

- ✿ 以符号#开始，表示本行#之后的内容为注释。

```
# this is my first Python Program  
prtint('Hi, Python!') # print a string
```

- ✿ 单行注释。
- ✿ 一条注释以不包含在字符串字面值内的井号 (#) 开头，并在物理行的末尾结束。
- ✿ 一条注释标志着逻辑行的结束。



Python程序的构成



■ 用多行字符串充当注释

- ✿ 包含在一对三引号 `''' ... '''` 或 `""" ... """` 之间，并且不属于任何语句的内容将被解释器认为是注释。

```
''' this is my first Python Program  
    print a string  
'''  
prtint('Hi, Python!')
```

- ✿ 可以看成是一种多行注释。



Python程序的构成



武汉大学
WUHAN UNIVERSITY

■ 添加注释后的三角形面积计算程序

```
triangle_area_commented.py - C:\Users\zhanghua\Desktop\mypython\p02\triangle_area_commented.py (3.11.4)
File Edit Format Run Options Window Help
1 # -*- coding: utf-8 -*-
2 """
3 输入三角形的三边长度，用海伦公式计算三角形的面积，并输出结果。
4 """
5 import math # 导入标准库模块math
6
7 # 输入数据
8 print("输入三角形的三边长度：") # 显示提示文本
9 a = float(input("a: "))
10 b = float(input("b: "))
11 c = float(input("c: "))
12
13 # 用海伦公式计算面积
14 h = (a + b + c)/2 # 计算三角形周长的一半
15 area = math.sqrt(h * (h - a) * (h - b) * (h - c)) # 计算面积
16
17 # 输出数据
18 print("三角形的面积为：", area)
```

Ln: 3 Col: 0



Python程序的构成



■ 用**编码声明**告知Python解释器程序源代码文件使用的字符编码。

✱ 编码声明必须独占一行，位于源代码文件的第一行或第二行注释中，一般采用下面的形式：

```
# -*- coding: <编码名称> -*-
```

➤ 编码名称必须是 Python 所认可的编码，例如utf-8、gb2312等。

```
# -*- coding: utf-8 -*-
```

✱ 在Unix/Linux系统上，编码声明可能会出现在第二行：

```
#!/usr/bin/env python3
```

```
# -*- coding: <编码名称> -*-
```

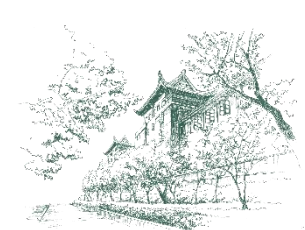
✱ 如果源文件中没有编码声明，则默认编码为 **UTF-8**。



武汉大学
WUHAN UNIVERSITY

03

编码规范



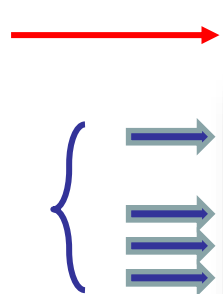
程序代码的书写规则



武汉大学
WUHAN UNIVERSITY

■ 补充编码规则说明

- ✱ 从第一列开始书写代码，前面不能有任何空格，否则会产生语法错误。
- ✱ 一般情况下，一条简单语句占一行。
- ✱ 可以用反斜杠（\）作为续行符，用于一个语句跨越多行的情况。
- ✱ 分号（;）用于在一行书写多条语句。
- ✱ 注释可以从任意位置开始。



```
import math

print("输入三角形的三边长度：")
a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

h = (a + b + c) / 2
area = math.sqrt(h * (h - a) * (h - b) * (h - c))

print("三角形的面积为：", area)
```




程序代码的书写规则



武汉大学
WUHAN UNIVERSITY

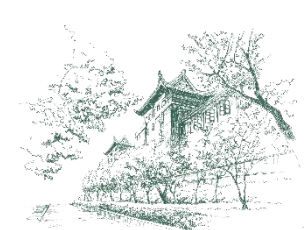
■ 补充编码规则说明

✿ 复合语句由头部语句和构造体语句块组成。

- 构造体语句块由一条或多条语句组成。
- 头部语句由相应的关键字开始，构造体语句块则为下一行开始的一行或多行缩进代码。
- 通常，缩进是相对头部语句缩进4个空格，也可以是任意多个空格，但同一构造体代码块的多条语句缩进的空格数必须一致对齐。如果语句不缩进，或缩进不一致，将导致编译错误。
- 如果条件语句、循环语句、函数定义和类定义比较短，可以放在同一行。

一条语句

```
''' 输入3个整数，输出最大数 '''  
a,b,c = eval(input(' 输入3个整数，用逗号隔开: '))  
if a>=b:  
    ← if a>=c:  
        ← print(f"最大数是{a}")  
    else:  
        print(f"最大数是{c}")  
else:  
    if b>=c:  
        print(f"最大数是{b}")  
    else:  
        print(f"最大数是{c}")
```



PEP 8编码规范



武汉大学
WUHAN UNIVERSITY

■ PEP 8是Python语言的一个官方代码风格指南。

✿ <https://peps.python.org/pep-0008/>

✿ PEP, Python Enhancement Proposal, Python增强提案

➤ Python社区正式文档

➤ 例如

– PEP 20 (Python之禅)

– PEP 484 (类型注解)

(1) 空格与缩进

- 使用 4 个空格进行缩进，不要使用 Tab 键。
- 每行不超过 79 个字符，注释和文档字符串每行不超过 72 个字符。
- 在二元运算符两侧各留一个空格，例如 `a = b + c`。

(2) 命名约定

- 变量、函数和属性应使用小写字母加下划线的方式命名，如 `my_variable_name`。
- 类名采用驼峰命名法 (CapWords)，即每个单词首字母大写，不使用下划线，如 `MyClassName`。
- 常量通常在模块级别定义，并且全部使用大写字母加下划线分隔，如 `MY_CONSTANT`。
- 私有变量或方法前加上单下划线，如 `_private_method`；对于希望避免子类访问的方法或属性，则前缀双下划线，如 `__private_method`。

(3) 注释与文档字符串

- 注释应当清晰明了，准确解释代码的功能。
- 文档字符串用于描述模块、函数、类和方法的目的和用法。

(4) 导入语句

- 每个导入单独一行。
- 将导入分为三个部分：标准库导入、相关第三方库导入、本地应用/库导入，每个部分之间用一个空行分隔，按照字母顺序排列。

(5) 表达式与语句

- 避免复杂的表达式，保持简单直接。
- 即使 `if` 语句等只包含一条语句，也应分开书写以增强可读性。



武汉大学
WUHAN UNIVERSITY

04

对象与变量



数据都是对象



武汉大学
WUHAN UNIVERSITY

- 在Python中，一切都是对象，意味着语言中的几乎所有实体都是对象，包括数据类型、函数、类、模块等。
- 对象是类的实例，而类定义了对象的属性和方法。
 - ✱ 例如，整数类 (int) 定义了
 - 整数的基本操作，比如加法、减法、乘法等，
 - 以及整数的属性（如二进制表示、字节长度等）。
 - ✱ 那么，每一个整数（例如12）就是int的一个实例，它有自己的值（12的值是12），也可以调用int中定义的方法，例如bit_length。

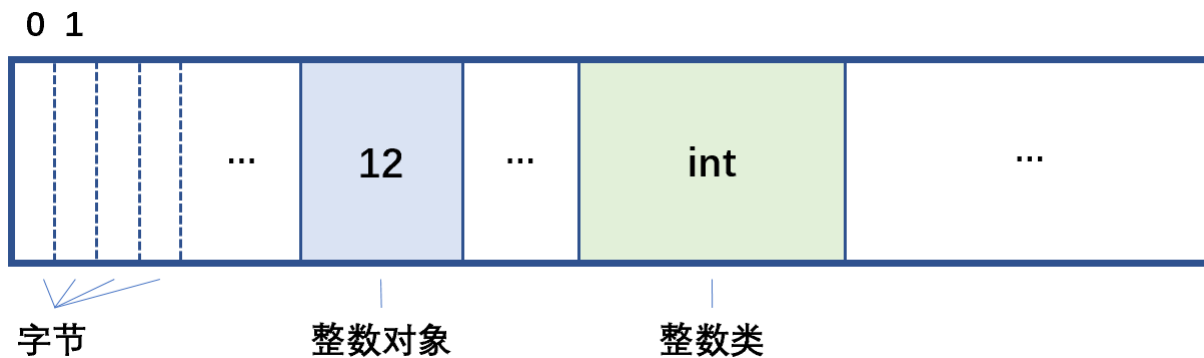


对象是一个内存实体



■ 对象本质上是一个内存实体。

✿ 如果把计算机的内存看成由无数个箱子组成的储物架（以字节为单位），对象就是储物架中不同大小的箱子。



✿ 图中整数12是一个对象，整数类int也是一个对象。

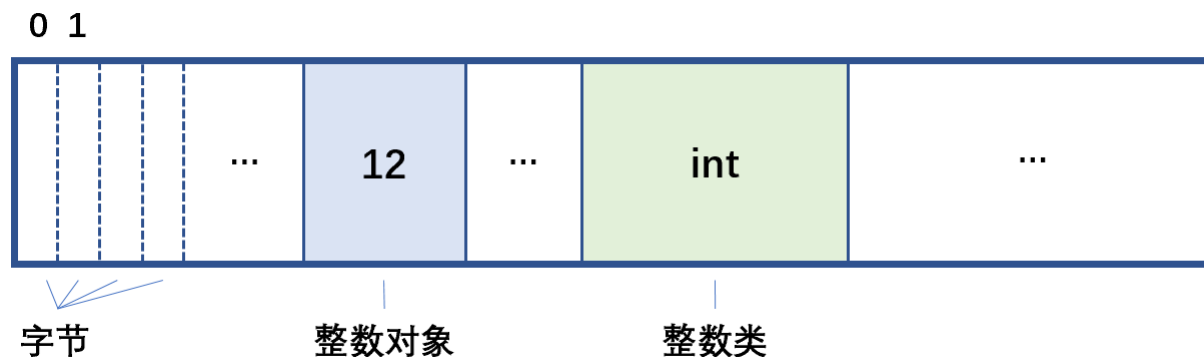


对象



对象都有标识、类型和值。

- 对象箱子的大小（占用的内存字节数）取决于对象所属的类型。
- 箱子在储物架上的位置（所占空间的第一个字节的编号，即内存地址）就唯一标识了对象。
- 箱子里的内容就是对象的值。





对象



武汉大学
WUHAN UNIVERSITY

对象的标识、类型和值可以通过内置函数id、type和print获取或输出。

```
>>> id(12)
140722381755528
>>> type(12)
<class 'int'>
>>> print(12)
12
>>> id(3.45)
1667134246096
>>> type(3.45)
<class 'float'>
>>> print(3.45)
3.45
>>> id('Python')
140722380497744
>>> type('Python')
<class 'str'>
>>> print('Python')
Python
```

```
>>> type(math)
<class 'module'>
>>> id(math)
2318711987800
>>> print(math)
<module 'math' (built-in)>
```

```
>>> type(print)
<class 'builtin_function_or_method'>
>>> id(print)
2318674436408
>>> print(print)
<builtin function print>
```

```
>>> type(int)
<class 'type'>
>>> id(int)
140728471278048
>>> print(int)
<class 'int'>
```

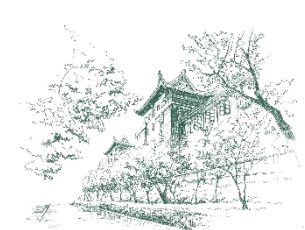


内置数据类型



- Python是一种强类型语言。
 - 每一个对象都是某一个类的实例，或者说对象都有类型，而且是不能改变的。
 - 类型决定了对象占用内存空间的大小、存储方式、以及能做的操作。
- 如果对象是数据型的，例如整数、浮点数和字符串等，其类型一般习惯上称为数据类型。
- 常用内置数据类型表→

名称	标识	可变性	字面值示例	简要说明
整数	int	不可变	12, -10, 0	表示整数值，支持任意大小的整数。
浮点数	float	不可变	3.14, -0.001, 2.0	表示带小数点的数字，支持科学计数法（如 1.23e-4）。
复数	complex	不可变	3+4j, 5.6j	表示复数，支持复数运算。
布尔值	bool	不可变	True, False	表示逻辑值，只有两个可能的值：True 或 False。
字符串	str	不可变	"hello", 'world'	表示文本数据，由字符序列组成，支持索引和切片操作。
字节串	bytes	不可变	b'hello', b'\x01\x02'	表示不可变的字节序列，通常用于处理二进制数据。
字节数组	bytearray	可变		表示可变的字节序列，功能与 bytes 类似，但内容可以修改。
列表	list	可变	[1, 2, 3], ['a', 'b']	表示有序的可变序列，可以包含任意类型的元素。
元组	tuple	不可变	(1, 2, 3), ('a', 'b')	表示有序的不可变序列，可以包含任意类型的元素。
集合	set	可变	{1, 2, 3}, {'a', 'b'}	表示无序且不重复的元素集合，支持集合运算（如并集、交集）。
冻结集合	frozenset	不可变		表示不可变的集合，功能与 set 类似，但内容不可修改。
字典	dict	可变	{'name': '于飞', 'age': 25}	表示键值对的集合，键必须是不可变类型（如字符串、整数）。
空类型	NoneType	不可变	None	表示空值或无值，通常用于初始化变量或表示函数无返回值。



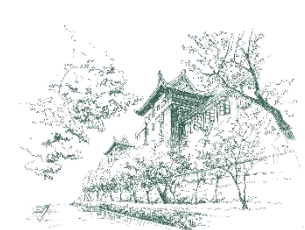
检测对象的类型



武汉大学
WUHAN UNIVERSITY

■ 可以用内置函数**isinstance**检测对象的类型。

```
>>> isinstance('Python', str)
True
>>> pi=3.14
>>> isinstance(pi, float)
True
```

创建对象



武汉大学
WUHAN UNIVERSITY

■ 创建对象的方式有以下几种：

- ✱ 直接写出字面值就是创建对象。
- ✱ 通过调用函数input，接收输入创建对象。
- ✱ 通过计算创建新对象。
 - 例如执行 $123 + 456$ 得到新对象579。
- ✱ 通过把数据类型名称当作函数调用，也可以创建对象。
 - 例如执行`int(123456)`，将创建一个新对象123456。

■ 创建对象后，如何引用它？



变量



- Python语言的变量是对象的标签，用来引用对象。
 - ✱ 这是Python语言的一大特点。
- 在形式上，变量是一个用户定义的名字。
- Python程序中的变量不需要提前声明，而是通过给它赋值来创建的，自创建开始变量就必须引用一个对象。
 - ✱ 例如，执行下面的语句后，变量sample被创建，并引用整数对象123。

```
sample = 123
```



变量与对象的关系

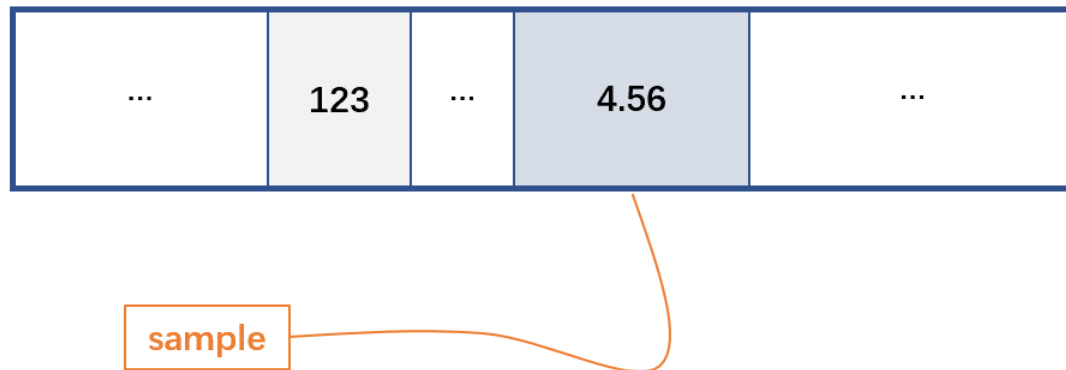


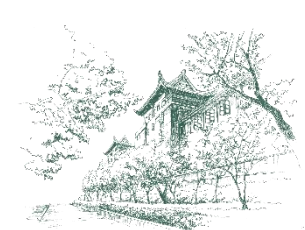
■ 变量只是对象的标签，可以引用任何类型的对象，且可以随时通过赋值把这个标签“贴到”另一个对象上。

✱ 例如，执行下面的语句后，sample引用浮点数对象4.56。

```
sample = 4.56
```

✱ 变量sample就是一个带有线的标签，线的另一头系着内存中的对象4.56。





变量与对象的关系



武汉大学
WUHAN UNIVERSITY

Python是动态类型语言。

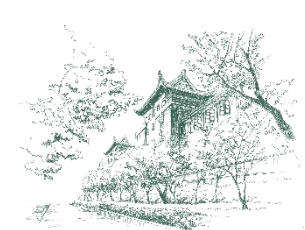
✿ 指的是变量可以随时重新赋值，从而改为引用另一个对象，新对象的类型可以与先前引用的对象的类型不同。

➤ 例如，变量`sample`起初引用整数`123`，然后引用浮点数`4.56`。

✿ 当然，多个变量可以引用同一个对象。

➤ 例如，执行下面的语句后，变量`sample`和`value`引用同一个浮点数`4.56`。

```
value = sample
print(id(sample)) # 输出2244234017712
print(id(value))  # 输出2244234017712
```



删除变量和对象

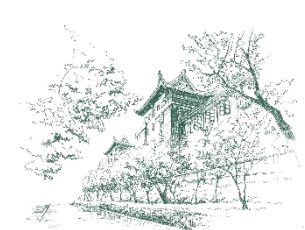


- 变量和它引用的对象是相互独立的。
- 可以用del语句删除不再需要的变量，但是该变量引用的对象不会被删除。
 - ✿ 例如，执行下面的语句后，变量value不存在了，但是它先前引用的4.56依然存在。

```
del value
print(value)    # NameError: name 'value' is not defined
print(sample)   # 输出4.56
print(id(sample)) # 输出2244234017712
```

■ 对象什么时候被删除呢？

- ✿ 当一个对象不再被任何变量引用，即对象的引用计数为0，Python会自动删除（或称为销毁）该对象，释放该对象占用的内存。



■ Python使用引用计数作为垃圾回收的主要机制之一。

✿ 每个对象都有一个引用计数，表示有多少变量或数据结构引用该对象。

- (1) 当对象被创建并赋值给变量时，引用计数为1。
- (2) 当对象被其他变量引用时，引用计数增加。
- (3) 当引用该对象的变量被删除或重新赋值时，引用计数减少。
- (4) 当引用计数变为0时，对象不再被任何变量引用，Python会自动销毁该对象并释放其占用的内存。

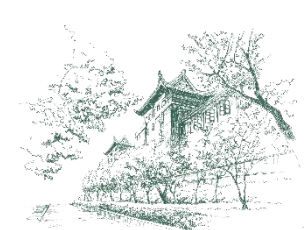
```
x = 12345 # 创建一个整数对象，引用计数为 1
y = x    # y 引用同一个整数对象，引用计数为 2
del x    # 删除变量 x，整数对象12345的引用计数减 1（引用计数为 1）
del y    # 删除变量 y，整数对象12345的引用计数减 1（引用计数为 0），该对象被销毁
```




武汉大学
WUHAN UNIVERSITY

05

数字类型



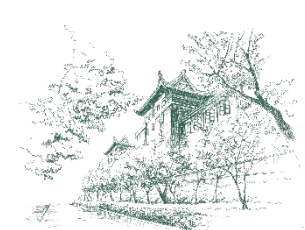
数字类型



武汉大学
WUHAN UNIVERSITY

Python的数字类型用于存储数值型数据。

- ✱ 整数 (int)
- ✱ 浮点数 (float)
- ✱ 复数 (complex)
- ✱ 此外，Python还支持布尔值 (bool)，它是整数类型的子类型。



整数类型



■ 整数类型用于表示整数值，可以是正数、负数或零。

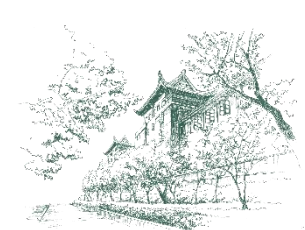
✱ Python的整数类型没有大小限制（仅受内存限制）。

✱ 特点：

- （1）支持任意大小的整数。
- （2）字面值可以是十进制、二进制（前缀**0b**或**0B**）、八进制（前缀**0o**或**0O**）或十六进制（前缀**0x**或**0X**）。

✱ 示例

```
a = 10    # 十进制
b = 0b1010 # 二进制，表示 10
c = 0o12   # 八进制，表示 10
d = 0xA    # 十六进制，表示 10
print(a, b, c, d) # 输出 10 10 10 10
```



浮点数类型



■ 浮点数类型用于表示带小数点的数字，对应数学中的实数。

✿ Python的浮点数遵循IEEE 754标准，使用双精度（64位）表示。

✿ 特点：

- （1）字面值可以用十进制小数形式或科学计数法。
- （2）由于浮点数的精度限制，可能存在精度误差。

✿ 示例

```
x = 3.14    # 普通浮点数
y = 2.5e-3  # 科学计数法，表示 0.0025
print(x, y) # 输出 3.14 0.0025
print(0.1 + 0.2) # 输出 0.30000000000000004
```



复数类型



武汉大学
WUHAN UNIVERSITY

■ 复数类型用于表示复数。

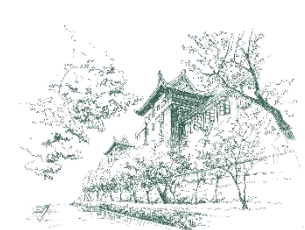
✱ 形式为 $a + bj$ ，其中 a 是实部， b 是虚部， j 表示虚数单位。

✱ 特点：

- (1) 复数的实部和虚部都是浮点数保存。
- (2) 支持复数的基本运算。

✱ 示例

```
z = 2 + 3j  # 复数
print(z)    # 输出 (2+3j)
print(z.real)  # 输出实部 2.0
print(z.imag)  # 输出虚部 3.0
```

布尔类型



■ 布尔类型用于表示逻辑值，只有两个可能的值：True和False。

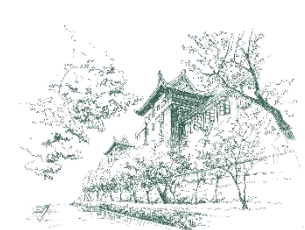
✿ 布尔类型是整数类型的子类型，True等价于1，False等价于0。

✿ 特点：

- (1) 用于条件判断和逻辑运算。
- (2) 可以与其他数字类型进行运算。

✿ 示例

```
is_valid = True
is_empty = False
print(is_valid + 1)  # 输出 2 (True 被当作 1)
print(is_empty * 10) # 输出 0 (False 被当作 0)
```

类型转换



int()函数

✿ 用于创建整数对象。它可以将其他类型的数据（如字符串、浮点数、布尔值）转换为整数。

✿ 语法：

`int(x, base=10)`

➤ **x**：要转换的值（可以是字符串、浮点数、布尔值等）。

➤ **base**：进制基数（默认为10，表示十进制）。

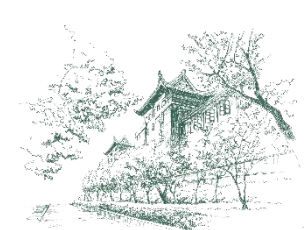
✿ 示例

```
# 从字符串创建整数
x = int("123")
print(x)  # 输出 123
```

```
# 从浮点数创建整数（截断小数部分）
y = int(3.14)
print(y)  # 输出 3
```

```
# 从布尔值创建整数
z = int(True)
print(z)  # 输出 1
```

```
# 从二进制字符串创建整数
w = int("1010", 2)
print(w)  # 输出 10
```



float()函数

✱ 用于创建浮点数对象。它可以将其他类型的数据（如字符串、整数、布尔值）转换为浮点数。

✱ 语法：

`float(x)`

➤ **x**：要转换的值（可以是字符串、整数、布尔值等）。

✱ 示例

```
# 从字符串创建浮点数
x = float("3.14")
print(x)  # 输出 3.14
```

```
# 从整数创建浮点数
y = float(10)
print(y)  # 输出 10.0
```

```
# 从布尔值创建浮点数
z = float(True)
print(z)  # 输出 1.0
```



complex()函数

✱ 用于创建复数对象。它可以将其他类型的数据（如字符串、整数、浮点数）转换为复数。

✱ 语法：

`complex(real, imag=0)`

➤ **real**：实部（可以是整数、浮点数、字符串等）。

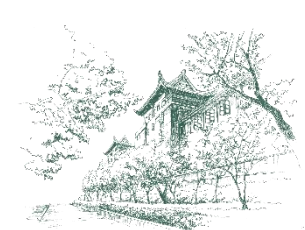
➤ **imag**：虚部（默认为 0）。

✱ 示例

```
# 从实部和虚部创建复数
x = complex(3, 4)
print(x)  # 输出 (3+4j)
```

```
# 从字符串创建复数
y = complex("5+6j")
print(y)  # 输出 (5+6j)
```

```
# 仅指定实部
z = complex(7)
print(z)  # 输出 (7+0j)
```



bool()函数

✱ 用于创建布尔值对象。它可以将其他类型的数据（如整数、浮点数、字符串）转换为布尔值。

✱ 语法：

`bool(x)`

➤ **x**：要转换的值（可以是数字、字符串、列表等）。

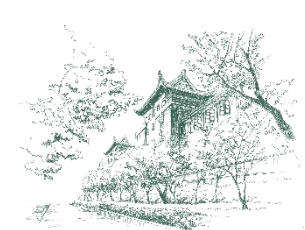
✱ 示例

```
# 从整数创建布尔值
x = bool(10)
print(x)  # 输出 True

# 从浮点数创建布尔值
y = bool(0.0)
print(y)  # 输出 False
```

```
# 从字符串创建布尔值
z = bool("Hello")
print(z)  # 输出 True

# 从空字符串创建布尔值
w = bool('')
print(w)  # 输出 False
```



类型转换



武汉大学
WUHAN UNIVERSITY

- 如果直接调用类型函数`int()`、`float()`、`complex()`和`bool()`，则分别创建整数0、浮点数0.0、复数0j和False。
- 注意，上述类型函数可以实现数据的类型转换，虽然称为“转换”，但是并不会改变参数x本身的类型。

```
x = 3.14
print(x) # 输出: 3.14
print(id(x)) # 输出: 2319208272208
print(type(x)) # 输出: <class 'float'>

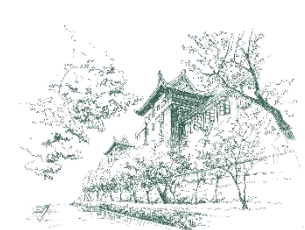
y = int(x)
print(y) # 输出: 3
print(id(y)) # 输出: 140724749898600
print(type(y)) # 输出: <class 'int'>

print(x) # 输出: 3.14
print(id(x)) # 输出: 2319208272208
print(type(x)) # 输出: <class 'float'>
```



数字对象的属性和方法

- 在Python中，数字对象具有一些属性和方法，用于操作和获取数字的相关信息。
- 要访问对象的属性和方法，需要使用圆点 (.) 运算符，也称为属性访问运算符。
 - ✱ 访问对象属性和方法的一般形式如下：
对象或变量.属性或方法名



数字对象的属性和方法



武汉大学
WUHAN UNIVERSITY

■ 整数

✿ 方法:

➤ `bit_length()`: 返回表示整数所需的二进制位数。

✿ 示例

```
x = 10  
print(x.bit_length()) # 输出 4 (10 的二进制是 1010, 需要 4 位)
```



数字对象的属性和方法



武汉大学
WUHAN UNIVERSITY

■ 浮点数

✿ 方法:

- `is_integer()`: 检查浮点数是否为整数（即小数部分是否为 0）。
- `as_integer_ratio()`: 返回一个元组(`numerator`, `denominator`), 表示浮点数的分数形式。其中, `numerator`是分子, `denominator`是分母。
- `hex()`: 返回一个表示浮点数的十六进制字符串。
- `fromhex()`: 将十六进制字符串转换为浮点数。

✿ 示例

```
x = 3.14
print(x.is_integer()) # 输出 False

y = 3.0
print(y.is_integer()) # 输出 True

z = 3.75
print(z.as_integer_ratio()) # 输出 (15, 4)
```

```
print(x.hex()) # 输出 0x1.91eb851eb851fp+1

w = float.fromhex("0x1.91eb851eb851fp+1")
print(w) # 输出 3.14
```



数字对象的属性和方法



复数

属性:

- **real**: 返回复数的实部。
- **imag**: 返回复数的虚部。

方法:

- **conjugate()**: 返回复数的共轭复数。

示例

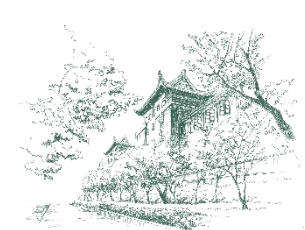
```
z = 2 + 3j
print(z.real)  # 输出 2.0
print(z.imag)  # 输出 3.0
print(z.conjugate())  # 输出 (2-3j)
```



武汉大学
WUHAN UNIVERSITY

06

表达式与运算



表达式与运算



武汉大学
WUHAN UNIVERSITY

- 在Python中，表达式和运算是紧密相关的概念。
- 表达式是由变量、常量、运算符和函数调用等组成的代码片段，而运算是表达式中的具体操作（如加法、减法、比较等）。
- Python支持多种类型的运算，包括算术运算、比较运算、逻辑运算、位运算等。



算术运算



武汉大学
WUHAN UNIVERSITY

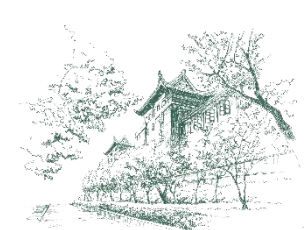
■ 算术运算用于执行基本的数学计算。

运算符	描 述	示 例	运算结果
+	加法	$10 + 5$	15
-	减法	$10 - 5$	5
*	乘法	$10 * 5$	50
/	除法	$10 / 5$	2.0
//	整除	$10 // 3$	3
%	取余	$10 \% 3$	1
**	幂运算	$2 ** 3$	8



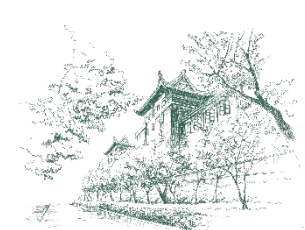
■ 比较运算用于比较两个值的大小或相等性，返回布尔值（True 或 False）。

运算符	描 述	示 例	运算结果
==	等于	10 == 5	False
!=	不等于	10 != 5	True
>	大于	10 > 5	True
<	小于	10 < 5	False
>=	大于等于	10 >= 5	True
<=	小于等于	10 <= 5	False



■ 逻辑运算用于组合多个条件，返回布尔值（True 或 False）。

运算符	描 述	示 例	运算结果
and	逻辑与	True and False	False
or	逻辑或	True or False	True
not	逻辑非	not True	False



位运算



■ 位运算直接对整数的二进制位进行操作。

运算符	描 述	示 例	运算结果
&	按位与	10 & 5	0
	按位或	10 5	15
^	按位异或	10 ^ 5	15
~	按位取反	~10	-11
<<	左移	10 << 1	20
>>	右移	10 >> 1	5



赋值运算



■ 赋值运算用于将运算结果赋值给变量。

✿ 简单赋值和复合赋值

运算符	描 述	示 例	等价于
=	赋值	<code>x = 10</code>	<code>x = 10</code>
+=	加后赋值	<code>x += 5</code>	<code>x = x + 5</code>
-=	减后赋值	<code>x -= 5</code>	<code>x = x - 5</code>
*=	乘后赋值	<code>x *= 5</code>	<code>x = x * 5</code>
/=	除后赋值	<code>x /= 5</code>	<code>x = x / 5</code>
//=	整出后赋值	<code>x //= 5</code>	<code>x = x // 5</code>
%=	取余后赋值	<code>x %= 5</code>	<code>x = x % 5</code>
**=	幂运算后赋值	<code>x **= 5</code>	<code>x = x ** 5</code>



赋值运算



链式赋值

✿ 在一条语句中把一个对象赋值给多个变量，通常用来初始化多个变量。

✿ 链式赋值的一般形式为：

变量名 = 变量名 = ... = 表达式（包括字面值）

✿ 示例：

```
a = b = c = 0  
print(a, b, c) # 输出: 0 0 0
```



赋值运算



武汉大学
WUHAN UNIVERSITY

■ 序列解包赋值

✱ 将可迭代对象（如列表、元组、字符串等）的元素直接赋值给多个变量。

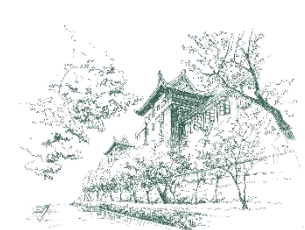
✱ 基本用法：

```
# 将元组中的值解包到变量
a, b, c = (1, 2, 3)
print(a, b, c) # 输出: 1 2 3
```

```
# 将元组中的值解包到变量
a, b, c = 1, 2, 3
print(a, b, c) # 输出: 1 2 3
```

✱ 可以用一种简单方式实现两个变量的交换：

```
a, b = 10, 20
print(a, b) # 输出: 10 20
a, b = b, a # 交换后 a = 20, b = 10
print(a, b) # 输出: 20 10
```

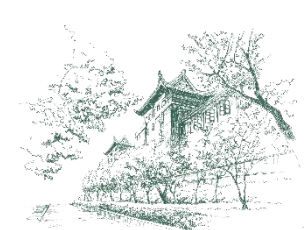
成员检测运算



武汉大学
WUHAN UNIVERSITY

■ 成员检测运算用于检查某个值是否存在于序列（如列表、字符串）中。

运算符	描 述	示 例	运算结果
in	存在于	'a' in 'abc'	True
not in	不存在于	'd' not in 'abc'	True



同一性检测运算

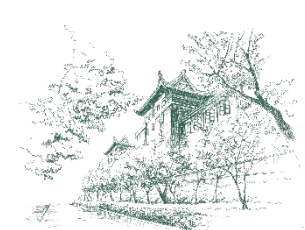


武汉大学
WUHAN UNIVERSITY

■ 同一性检测运算用于比较两个对象的内存地址，判断它们是否是同一个对象。

运算符	描 述	示 例	运算结果
is	是同一个对象	x is y	True 或 False
is not	不是同一个对象	x is not y	True 或 False

✿ 注意，“is”用于判断两个变量是否引用同一个对象，而比较运算“==”用于判断两个对象的值是否相等。



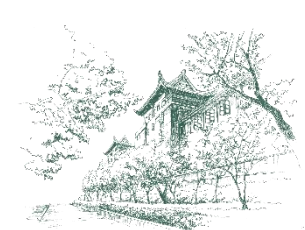
关于表达式的进一步说明



■ 操作数、运算符和圆括号按一定规则连接组成表达式。

■ Python 表达式的书写规则：

- ✿ (1) 表达式从左到右在同一个基准上书写。例如，数学公式 $a^2 + b^2$ 应该写为：
`a ** 2 + b ** 2`。
- ✿ (2) 乘号不能省略。例如，数学公式 ab （表示 a 乘以 b ）应写为：`a * b`。
- ✿ (3) 圆括号用来改变计算顺序，必须成对出现，而且只能使用圆括号。
- ✿ (4) 圆括号可以嵌套使用。



关于表达式的进一步说明

■ 与数学中的计算公式类似，Python的一个表达式中也可以出现多种运算。

■ Python 表达式的求值顺序遵循以下规则：

- ✱ (1) 括号优先：括号内的表达式先求值。
- ✱ (2) 运算符优先级：优先级高的运算符先求值。
- ✱ (3) 运算符结合性：多个相同优先级的运算符根据结合性从左到右求值，或者从右往左求值。

```
result = 10 + 5 * 2 # 先计算 5 * 2，再计算 10 + 10
print(result) # 输出 20
result = (10 + 5) * 2 # 先计算 10 + 5，再计算 15 * 2
print(result) # 输出 30
```

```
print(2 ** 2 ** 3) # 先计算 2 ** 3，再计算 2 ** 8，输出 256
print((2 ** 2) ** 3) # 输出 64
```



运算符优先级



常用运算符→

- 优先级从高到低排列。
- 一般，运算符的结合性是左结合，但单目运算符（只需要一个操作数的运算符，如正号、负号、按位取反、逻辑非）和幂运算符的结合性是右结合。

运算符	描 述
()	括号
.	属性访问
**	幂运算
+x, -x, ~x	正号, 负号, 按位取反
*, /, //, %	乘, 除, 整除, 取余
+, -	加法, 减法
<<, >>	左移, 右移
&	按位与
^	按位异或
	按位或
==, !=, >, <, >=, <=	等于, 不等于, 大于, 小于, 大于等于, 小于等于
is, is not	同一性检测运算
in, not in	成员检测运算
not	逻辑非
and	逻辑与
or	逻辑或
=, +=, -=, *=, /=, //=, **=	赋值运算



关于表达式的进一步说明

■ 每个表达式都会返回一个值，这个值可以是：

- ✱ (1) 数字（如整数、浮点数）
- ✱ (2) 字符串
- ✱ (3) 布尔值（True 或 False）
- ✱ (4) 对象（如列表、字典）
- ✱ (5) None（表示空值）

```
x = 10 + 5 # 表达式返回 15  
y = "Hello" + "World" # 表达式返回 "HelloWorld"  
z = x > 20 # 表达式返回 False
```



数学运算内置函数

■ 在Python中，有若干内置函数可以用于数字，完成数学运算。

✿ 常用的有：

函数	描述	语法	示例	输出
<code>abs()</code>	返回数字的绝对值	<code>abs(x)</code>	<code>abs(-10)</code>	10
<code>round()</code>	对浮点数进行四舍五入	<code>round(x[, n])</code>	<code>round(3.14159, 2)</code>	3.14
<code>pow()</code>	返回 x 的 y 次幂	<code>pow(x, y[, z])</code>	<code>pow(2, 3)</code>	8
<code>divmod()</code>	返回商和余数	<code>divmod(x, y)</code>	<code>divmod(10, 3)</code>	(3, 1)



标准库math模块



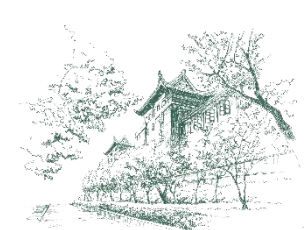
■ 提供了许多数学函数和常量，用于执行常见的数学运算。

✿ 数学常量

- `math.pi`: 返回圆周率 π 的值。
- `math.e`: 返回自然常数 e 的值。

✿ 数学函数

- `math.sqrt(x)`: 返回数字 x 的平方根。
- `math.pow(x, y)`: 返回 x 的 y 次幂。
- `math.log10(x)`: 返回数字 x 的以10为底的对数。
- `math.sin(x)`: 返回数字 x （注意是弧度）的正弦值。
- `math.ceil(x)`: 返回大于或等于数字 x 的最小整数。
- `math.floor(x)`: 返回小于或等于数字 x 的最大整数。



练习：算术运算表达式



■ 用Python表达式来表示一些数学表达式。

(1) 线性方程： $y = 2x + 3$

(2) 二次方程： $y = ax^2 + bx + c$

(3) 圆的面积： $A = \pi r^2$

(4) 两点间距离： $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

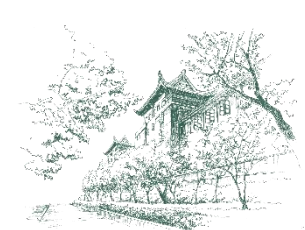
(5) 二次方程的根： $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$



武汉大学
WUHAN UNIVERSITY

07

字符串类型



字符串类型



■ 字符串类型 (str)

- ✿ 文本数据在Python中用字符串对象来存储。
- ✿ 字符串类定义了诸多方法对文本数据进行各种处理。

■ 字符串的字面值

- ✿ 用一对引号括起来的若干字符，可以是一对单引号、一对双引号、一对三单引号 and 一对三双引号。
- ✿ 用三单引号和三双引号的字符串是多行字符串，即输入字符串时可以直接换行。



字符串字面值



字符串的字面值示例

```
sentence = '这是一个单行字符串。'  
introduction = "I'm from China."  
multi_line_text = '''这是一个多行字符串，  
这里是第二行，  
第三行在这里。'''  
menu = """  
    学生成绩管理  
=====
```

- 【1】 登记成绩
- 【2】 浏览成绩
- 【3】 查询成绩
- 【4】 修改成绩
- 【5】 删除成绩

```
=====
```

```
"""
```

```
  
print(sentence)  
print(introduction)  
print(multi_line_text)  
print(menu)
```

这是一个单行字符串。
I'm from China.
这是一个多行字符串，
这里是第二行，
第三行在这里。

学生成绩管理

- ```
=====
```
- 【1】 登记成绩
  - 【2】 浏览成绩
  - 【3】 查询成绩
  - 【4】 修改成绩
  - 【5】 删除成绩
- ```
=====
```




转义字符

- 在Python中，转义字符是一些特殊的字符序列，用于表示无法直接输入或显示的字符（如换行符、制表符等）。
- 转义字符以反斜杠（\）开头，后跟一个或多个字符。
- 常用的转义字符及其含义：

转义字符	含义	转义字符	含义
\n	换行符	\r	回车符
\t	制表符 (Tab)	\f	换页符
\\	反斜杠	\ooo	八进制数 ooo 表示的 ASCII 字符
\'	单引号	\xhh	十六进制数 hh 表示的 ASCII 字符
\"	双引号	\uxxxx	4 位十六进制数 xxxx 表示的 Unicode 字符
\b	退格符	\Uxxxxxxxx	8 位十六进制数 xxxxxxxx 表示的 Unicode 字符



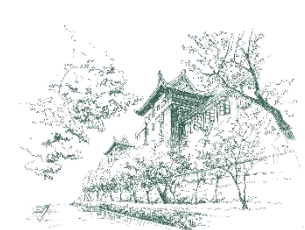
转义字符



转义字符示例

```
print("Hello\nWorld")
print("Name:\tAlice")
print("Age:\t25")
print("C:\\path\\to\\file")
print('It\'s me')
print("She said, \"Hi!\")
print("Hello\bWorld")
print("Hello\rWorld")
print("\110\145\154\154\157") # 八进制, 输出 "Hello"
print("\x48\x65\x6c\x6c\x6f") # 十六进制, 输出 "Hello"
# 表示多语言字符
print("Hello, \u4E16\u754C!") # 输出 Hello, 世界!
# 表示表情符号
print("I'm happy \U0001F60A") # 输出 I'm happy ☺
```

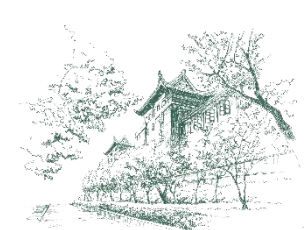
```
Hello
World
Name:   Alice
Age:    25
C:\path\to\file
It's me
She said, "Hi!"
HellWorld
World
Hello
Hello
Hello, 世界!
I'm happy ☺
```

■ 原始字符串

- ✿ 如果希望字符串中的转义字符不被解释，可以使用原始字符串。
- ✿ 原始字符串在字符串前加r或R。

```
print(r"Hello\nWorld")  # 输出 Hello\nWorld  
print(R"C:\path\to\file")  # 输出 C:\path\to\file
```



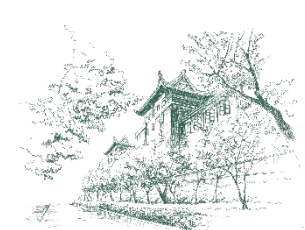
■ 字符串拼接

- ✿ 在Python中，字符串拼接是将多个字符串合并为一个字符串的操作。
- ✿ Python提供了多种方式来实现在字符串拼接。
- ✿ “+” 运算符可以直接将两个字符串拼接起来，创建一个新字符串。

```
s1 = "Hello"  
s2 = "World"  
s3 = s1 + ", " + s2  
print(s3)  # 输出 Hello, World
```

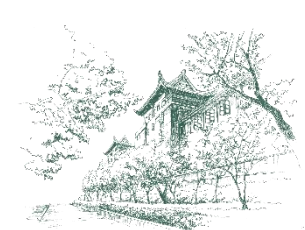
- ✿ 注意，参与拼接的必须是两个字符串。可以用函数str把数字转换为字符串，再与字符串拼接。

```
age = 18  
s = "我今年" + str(age) + "岁。"  
print(s)  # 输出：我今年18岁。
```



字符串格式化

- 在Python中，字符串格式化是将变量或表达式嵌入到字符串中的过程。
- Python提供了多种字符串格式化的方式：
 - ✿ f-string（格式化字符串字面值）
 - ✿ 字符串的format方法
 - ✿ 内置函数format
 - ✿ %格式化
- ✿ 每种方式都有其特点和适用场景。
- ✿ 在大多数情况下推荐使用 f-string，如果要做复杂格式化，建议使用字符串的format方法。不推荐另外两种方法。



f-string



■ f-string, 格式化字符串字面值, 或称为f字符串

- ✱ 是标注了'f'或'F'前缀的字符串字面值。
- ✱ 这种字符串可包含替换字段, 即以{}标注的表达式。
- ✱ 普通字符串字面值只是常量, 而格式化字符串字面值则是可在运行时求值的表达式。

■ 它允许在字符串中直接嵌入变量或表达式, 语法简单且可读性好。



f-string



f-string语法

f"字符串{表达式}字符串"

✳ 一对大括号中的表达式可以是字面值、变量、表达式、函数调用等。

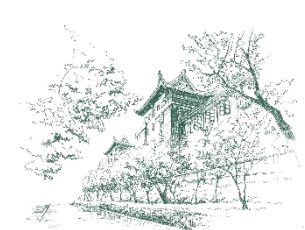
```
name = "Alice"
age = 25
s1 = f"My name is {name} and I am {age} years old."
print(s1) # 输出 My name is Alice and I am 25 years old.
```

```
x = 10
y = 20
s2 = f"The sum of {x} and {y} is {x + y}."
print(s2) # 输出 The sum of 10 and 20 is 30.
```

```
s3 = f"The result is {10 * 5}."
print(s3) # 输出 The result is 50.
```

```
name = "Alice"
s4 = f"My name is {name.upper()}."
print(s4) # 输出 My name is ALICE.
```

程序执行时，将用{}中表达式的值替换掉{表达式}整体，创建字符串。



f-string



f-string语法

- ✿ 可以在{}中使用**格式化规范**来格式化数字，包括控制对齐和填充。
- ✿ 格式化规范通过在{}中使用冒号 “:” 后跟格式化字符串来定义。
- ✿ 格式化规范的语法为：
{表达式:格式化字符串}

规 范	说 明	示 例
:<宽度	在指定宽度内左对齐	f"{'Alice':<10}"
:>宽度	在指定宽度内右对齐	f"{'Alice':>10}"
:^宽度	在指定宽度内居中对齐	f"{'Alice':^10}"
::精度	控制浮点数精度或字符串截断长度	f"{3.1415926:.2f}"
:,	千位分隔符	f"{1000000:,}"
:+	显示正负号	f"{10:+}"
:%	百分比格式	f"{0.25:%}"



f-string



■ 用f-string格式化数字示例

```
# 左对齐
s = f"Name: {'Alice':<10}"
print(s)  # 输出 Name: Alice

# 右对齐
s = f"Name: {'Alice':>10}"
print(s)  # 输出 Name:      Alice

# 浮点数精度
pi = 3.1415926
s = f"Pi: {pi:10.2f}"
print(s)  # 输出 Pi:      3.14

# 千位分隔符
s = f"Population: {1000000:,,}"
print(s)  # 输出 Population: 1,000,000

# 正负号和百分比
print(f"年增长率: {0.056:+.1%}")  # 输出 年增长率: +5.6%
```




字符串的format方法

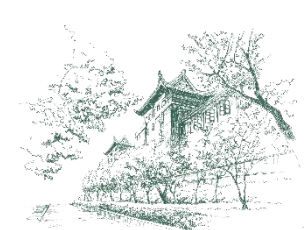
■ 字符串的format方法

- ✿ 是一种传统的字符串格式化工具，允许将变量或表达式插入到字符串中，并支持多种格式化选项。
- ✿ 其基本用法是通过占位符{}将变量或表达式插入到字符串中。

```
name = "Alice"  
age = 25  
s = "My name is {} and I am {} years old.".format(name, age)  
print(s)  # 输出 My name is Alice and I am 25 years old.
```

- ✿ 可以通过索引指定占位符的位置。

```
s = "{1} is {0} years old.".format(25, "Alice")  
print(s)  # 输出 Alice is 25 years old.
```



字符串的format方法

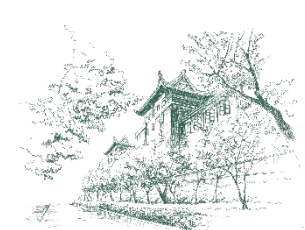


武汉大学
WUHAN UNIVERSITY

■ 字符串的format方法

✱ 可以通过关键字参数指定占位符的名称。

```
s = "My name is {name} and I am {age} years old.".format(name="Alice", age=25)
print(s)  # 输出 My name is Alice and I am 25 years old.
```



字符串的format方法



■ 用字符串的format方法格式化数字

✿ 字符串的format方法支持多种数字格式化选项，包括精度、对齐、填充等。

✿ (1) 格式化整数

- :d: 将数字格式化为整数。
- :x: 将数字格式化为十六进制。
- :b: 将数字格式化为二进制。

```
x = 255
print("Decimal: {:d}".format(x)) # 输出 Decimal: 255
print("Hex: {:x}".format(x))    # 输出 Hex: ff
print("Binary: {:b}".format(x)) # 输出 Binary: 11111111
```



字符串的format方法

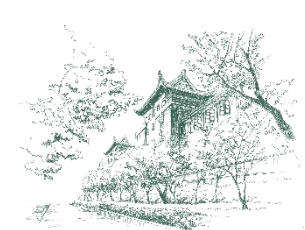


■ 用字符串的format方法格式化数字

✿ (2) 格式化浮点数

- `:.nf`: 保留 `n` 位小数。
- `:.n%`: 将数字转换为百分比，并保留 `n` 位小数。

```
pi = 3.1415926
print("Pi is approximately {:.2f}".format(pi)) # 输出 Pi is approximately 3.14
print("Percentage: {:.1%}".format(0.25)) # 输出 Percentage: 25.0%
```



字符串的format方法



■ 用字符串的format方法格式化数字

✿ (3) 对齐和填充

- `:%<n`: 左对齐, 宽度为 `n`。
- `:%>n`: 右对齐, 宽度为 `n`。
- `:%^n`: 居中对齐, 宽度为 `n`。
- `:%x`: 指定字符`x`为填充字符。

✿ (4) 千位分隔符

- `%,`: 添加千位分隔符。

```
x = 42
print("Left: {:<10}".format(x))    # 输出 Left: 42
print("Right: {:>10}".format(x))   # 输出 Right:          42
print("Center: {:^10}".format(x))  # 输出 Center:         42
print("Fill: {:0>10}".format(x))   # 输出 Fill: 0000000042

x = 1000000
print("Formatted: {:,}".format(x)) # 输出 Formatted: 1,000,000
```



用内置函数操作字符串

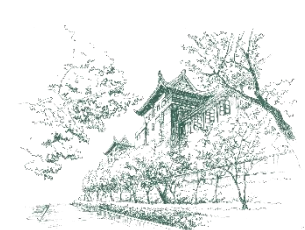
■ 在Python中，有许多内置函数可以用于操作字符串。

✿ len()函数

➤ 功能：返回字符串的长度（字符数）。

```
s = "Hello"
print(len(s))  # 输出 5

w = "武汉大学是一所美丽的大学。"
print(len(w))  # 输出 13
```



用内置函数操作字符串



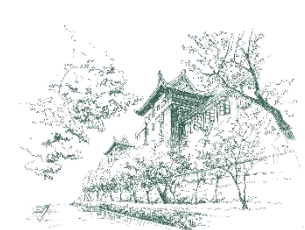
■ 操作字符串的内置函数

✿ str()函数

➤ 功能：将其他类型的数据转换为字符串。

```
x = 123
s = str(x)
print(s)  # 输出 "123"
print(str(4.56))  # 输出 4.56
print(str(7 + 8j))  # 输出 (7+8j)
print(str(12 > 3))  # 输出 True

name = "Alice"
age = 25
s = "My name is " + name + " and I am " + str(age) + " years old."
print(s)  # 输出 My name is Alice and I am 25 years old.
```

用内置函数操作字符串



■ 操作字符串的内置函数

✿ ord()函数

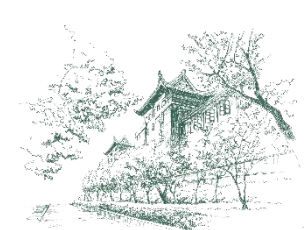
➤ 功能：返回字符的Unicode码点（编码值）。

✿ chr()函数

➤ 功能：返回 Unicode 码点对应的字符。

```
print(ord('A')) # 输出 65
print(ord('张')) # 输出 24352

print(chr(66)) # 输出 B
print(chr(21326)) # 输出 华
```



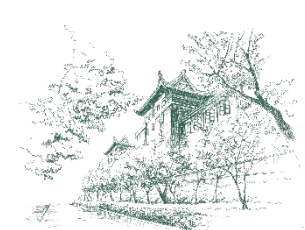
■ 操作字符串的内置函数

✿ eval()函数

➤ 功能：将字符串作为Python表达式执行。

```
s = "3 + 5"  
result = eval(s)  
print(result)  # 输出 8
```

```
values = "12, 34.5"  
v1, v2 = eval(values)  # 相当于 v1, v2 = 12, 34.5  
print(v1, v2)  # 输出 12 34.5
```



用内置函数操作字符串



■ 操作字符串的内置函数

✱ eval()函数的应用举例

➤ 一次输入三角形的三边长度。

```
a, b, c = eval(input("输入三角形的三边长度（用逗号隔开）："))  
print(f"a={a}, b={b}, c={c}")
```

```
输入三角形的三边长度（用逗号隔开）： 3,4,5  
a=3, b=4, c=5
```



武汉大学
WUHAN UNIVERSITY

08

实践指导



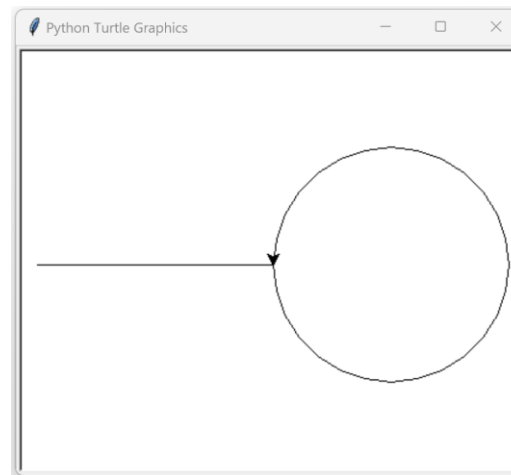
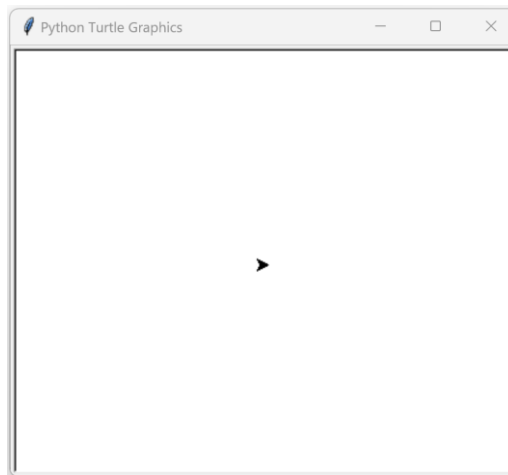
案例1：turtle绘图



turtle

- ✿ 标准库中turtle模块可以用来绘制图形和创建图形交互界面。
- ✿ 它通过控制一个“海龟”在屏幕上移动来绘制图形，非常适合初学者学习编程和图形绘制。

```
IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
Python 3.11.4 | packaged by Anaconda, Inc. | (main, Jul 5 2023, 13:38:37) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more
information.
>>> import turtle
>>> pen = turtle.Turtle()
>>> pen.backward(200)
>>> pen.forward(200)
>>> pen.right(90)
>>> pen.circle(100)
>>>
```



- 默认情况下，turtle使用的是一个以窗口中心为原点(0, 0)，向右为X轴正方向，向上为Y轴正方向的笛卡尔坐标系。
- 创建画笔后，海龟的头是朝向右的。

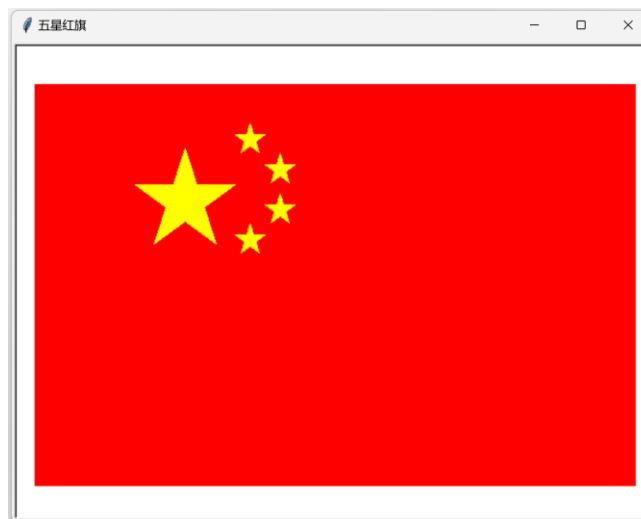


案例1：turtle绘图



武汉大学
WUHAN UNIVERSITY

■ 用turtle绘制五星红旗（非标准的示意图）

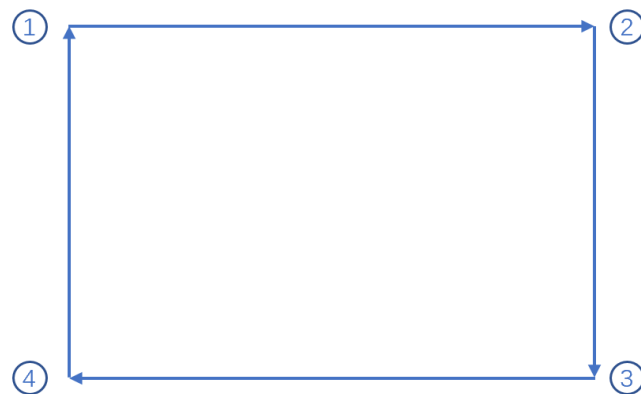




案例1：turtle绘图

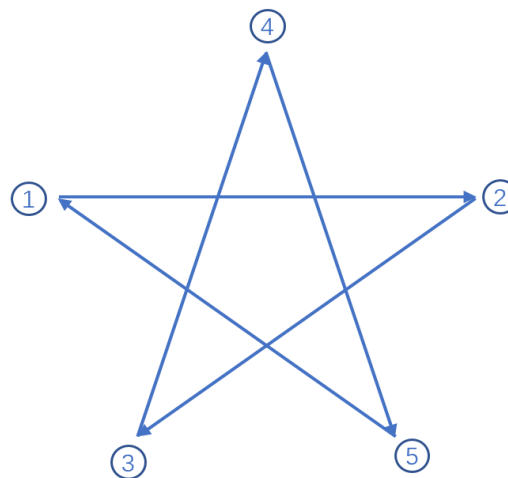


画法



(a)

画四条边，依次连接4个点，即1-2、2-3、3-4、4-1，就构成了矩形。



(b)

从第1点开始，按1-2-3-4-5-1的顺序，画5条边，在第2、3、4、5点处各右转 144° （因为 $360^\circ / 5 = 72^\circ$ ，而五角星需要跳过两个顶点，所以 $72^\circ * 2 = 144^\circ$ ），就构成了五角星。



案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (1) 导入turtle模块

```
import turtle
```

✿ (2) 创建画布和画笔

```
# 创建画布
screen = turtle.Screen()
screen.setup(width=640, height=480) # 设置画布大小
screen.title("五星红旗") # 设置画布的标题

# 创建画笔
pen = turtle.Turtle()
```



案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (3) 设置和控制画笔

```
# 设置画笔属性
pen.speed(0)  # 设置绘制速度为最快速度
pen.penup()

# 绘制红色背景
pen.color("red")
pen.begin_fill()
pen.goto(-300, 200)  # 左上角
pen.pendown()
pen.goto(300, 200)   # 右上角
pen.goto(300, -200)  # 右下角
pen.goto(-300, -200) # 左下角
pen.goto(-300, 200)  # 回到左上角
pen.end_fill()
pen.penup()
```



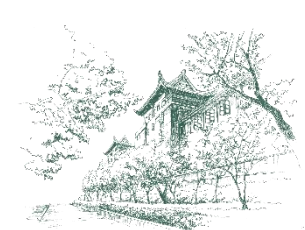
案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (3) 设置和控制画笔

- **pen.pensize(width)**: 设置画笔宽度。**width**表示画笔的粗细，单位为像素，可以是整数或浮点数。
- **pen.pencolor(color)**: 设置画笔颜色。**color**参数可以是颜色名称（如"red"、"blue"），也可以是RGB元组（如(255, 0, 0)表示红色）。
- **pen.speed(speed)**: 画笔绘制的速度。**speed**为[0, 10]间的整数画笔。
- **pen.forward(distance)**: 向前移动指定距离。**distance**表示海龟向前移动的距离，单位为像素，可以是整数或浮点数。如果**distance**为负数，表示向反方向移动。
- **pen.backward(distance)**: 向后移动指定距离。
- **pen.right(angle)**: 向右转指定角度。**angle**表示旋转的角度，单位为度，可以是整数或浮点数。
- **pen.left(angle)**: 向左转指定角度。



案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (3) 设置和控制画笔

- `pen.goto(x, y)`: 移动到指定坐标。
- `pen.penup()`: 抬起画笔（移动时不绘制）。
- `pen.pendown()`: 放下画笔（移动时绘制）。
- `pen.fillcolor(color)`: 指定填充颜色。
- `pen.begin_fill()`: 记录当前路径的起点，并在后续绘制过程中跟踪路径。
- `pen.end_fill()`: 用当前设置的颜色填充闭合路径内的区域。
- `pen.circle(radius)`: 绘制指定半径的圆。
- `pen.reset()`: 重置画布和画笔。
- `pen.hideturtle()`: 隐藏海龟。
- `pen.showturtle()`: 显示海龟。



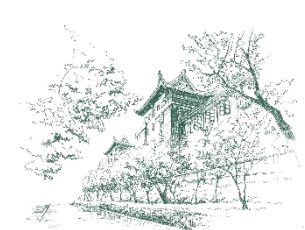
案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (3) 设置和控制画笔

```
# 绘制大五角星
pen.color("yellow")
pen.goto(-200, 100) # 大五角星第一顶点的位置
pen.pendown()
pen.begin_fill()
pen.forward(100)
pen.right(144)
pen.forward(100)
pen.right(144)
pen.forward(100)
pen.right(144)
pen.forward(100)
pen.right(144)
pen.forward(100)
pen.right(144)
pen.end_fill()
pen.penup()
```



案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (3) 设置和控制画笔

```
# 绘制第一颗小五角星
pen.goto(-100, 150) # 第一颗小五角星第一顶点的位置

# 绘制第二颗小五角星
pen.goto(-70, 120) # 第二颗小五角星第一顶点的位置

# 绘制第三颗小五角星
pen.goto(-70, 80) # 第三颗小五角星第一顶点的位置

# 绘制第四颗小五角星
pen.goto(-100, 50) # 第四颗小五角星第一顶点的位置
```

- 练习：程序中定位了4颗小五角星的第一顶点的位置，参考绘制大五角星的语句补全绘制4颗小五角星的语句。
- 注意：国旗有严格的规格要求。



案例1：turtle绘图



■ 用turtle绘制五星红旗

✿ (4) 隐藏画笔和关闭画布

```
# 结束  
pen.hideturtle()  
turtle.done() # 保持窗口打开
```

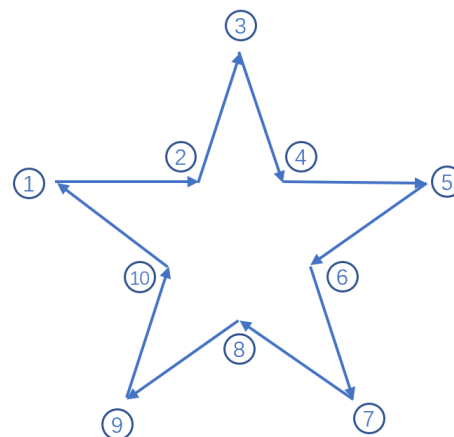
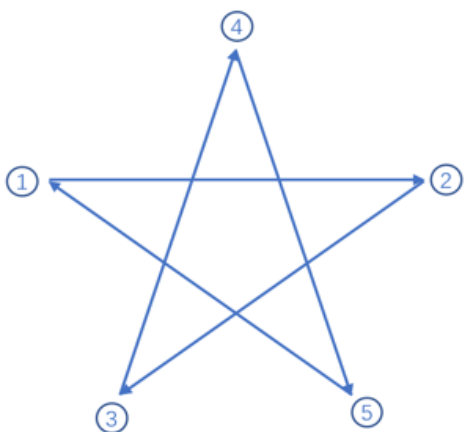



案例1：turtle绘图



改进提示

- ✿ 在程序的语句`pen.begin_fill()`和`pen.end_fill()`之间，要画一个封闭形状，才可以对该封闭形状进行填充。
- ✿ 这个程序采用的五角星画法虽然笔划少，笔划却有交叉，不是一个理想五角星封闭形状。在某些环境中运行该程序，五角星的中间不会填充黄色。
- ✿ 建议采用下面新的画法，然后自行设计和编写程序，绘制无瑕疵的五星红旗。





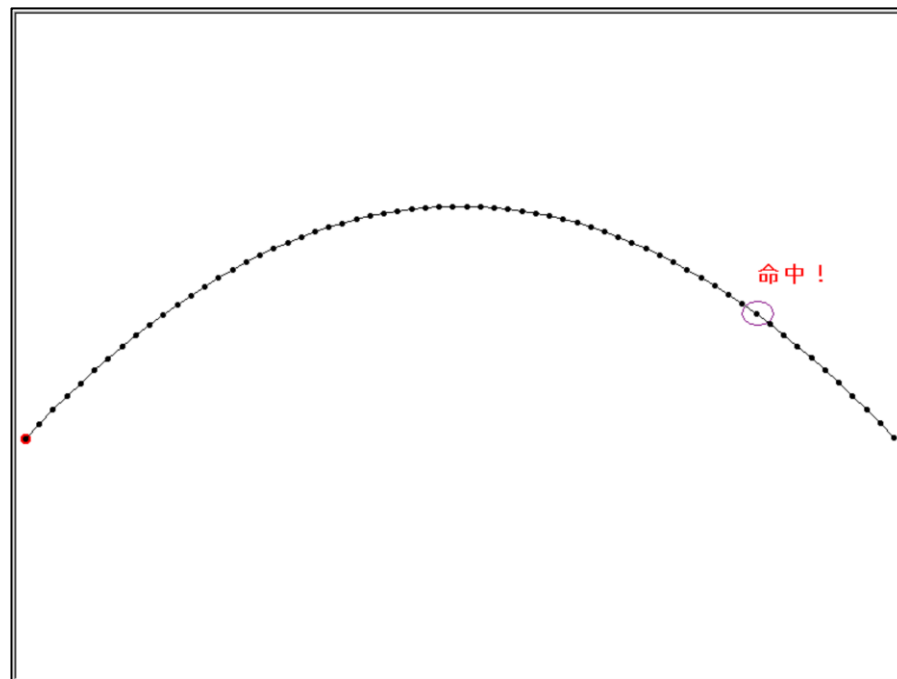
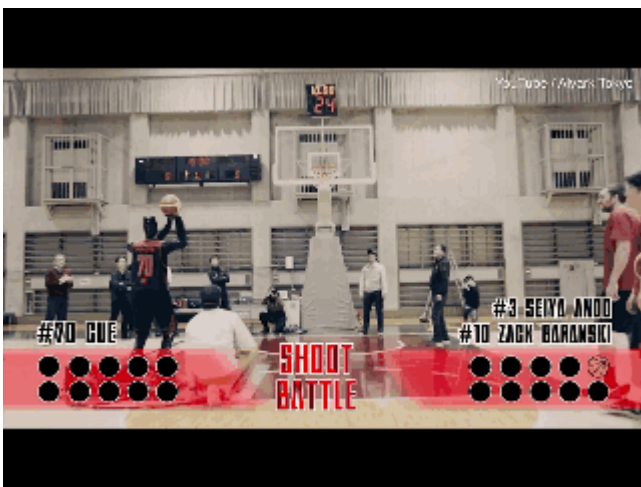
案例2：投篮模拟



武汉大学
WUHAN UNIVERSITY

问题描述

- ✱ 编写一个机器人投篮模拟程序。
- ✱ 根据已知的篮圈高度、机器人投篮高度和投篮距离，计算投篮命中的投篮角度和速度，并模拟投篮轨迹。





案例2：投篮模拟

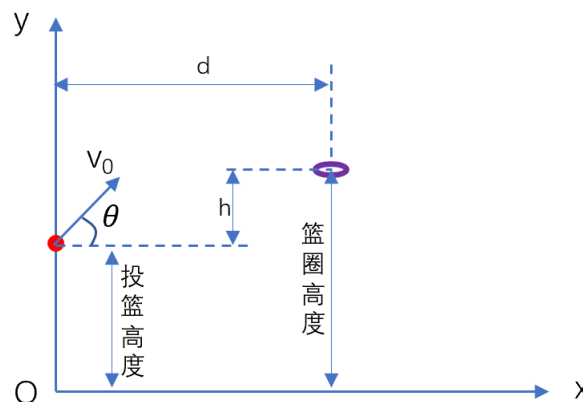
■ 采用IPO方法设计程序，投篮模拟程序包括以下功能：

✿ (1) 输入参数

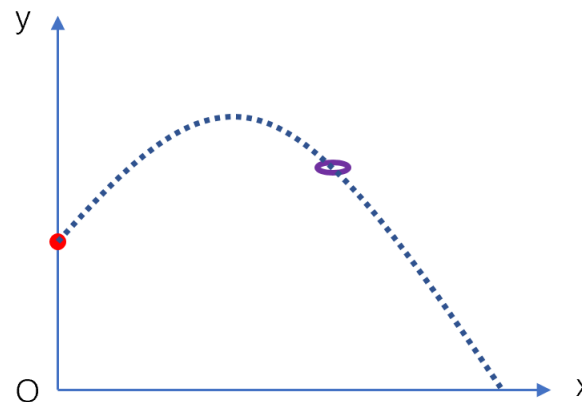
- 篮圈高度（米）。
- 机器人投篮高度（米）。
- 投篮距离（米）。

✿ (2) 计算投篮角度和速度

- 建立投篮模型的过程→



(a)



(b)

- 使用物理公式计算投篮角度和出手速度（假设忽略空气阻力）。

投篮角度 $\theta = \arctan\left(\frac{h + \sqrt{h^2 + d^2}}{d}\right)$ ，其中 h 是篮圈高度与机器人投篮高度的差值， d 是投篮距离。

出手速度 $v_0 = \sqrt{\frac{g \cdot d^2}{2 \cdot \cos^2(\theta) \cdot (d \cdot \tan(\theta) - h)}}$ ，其中 $g = 9.81 \text{ m/s}^2$ 。



案例2：投篮模拟

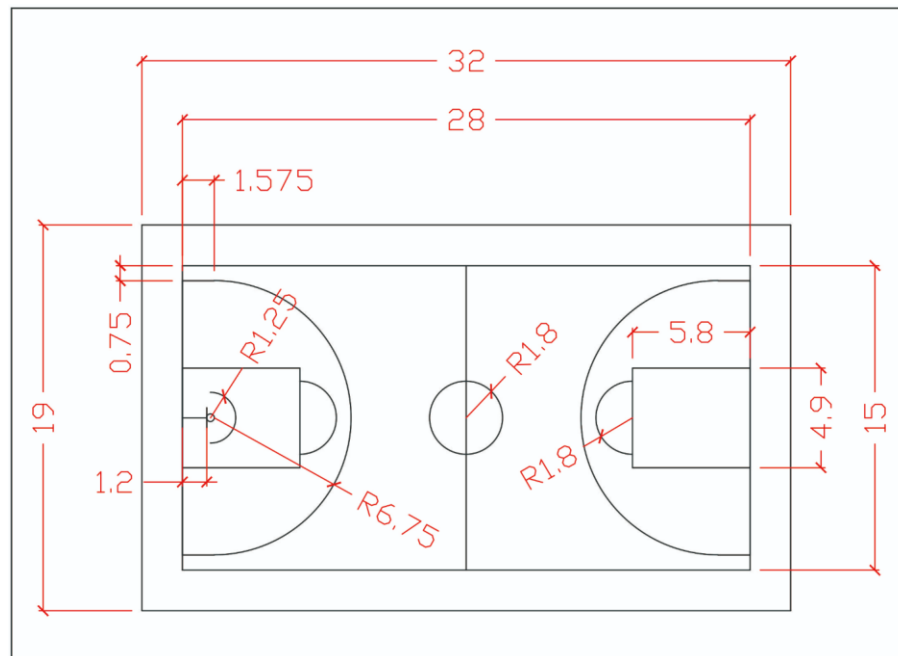


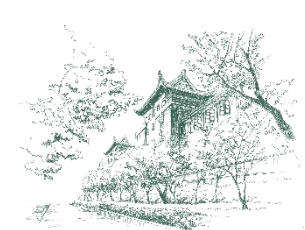
■ 采用IPO方法设计程序，投篮模拟程序包括以下功能：

✿ (3) 输出结果

- 输出计算出的投篮角度（以度为单位）。
- 输出计算出的出手速度（以米/秒为单位）。

✿ 篮球场的规格→





案例2：投篮模拟



武汉大学
WUHAN UNIVERSITY

编写投篮模拟程序

```
import math

# 输入参数
hoop_height = float(input("请输入篮圈高度（米）："))
robot_height = float(input("请输入机器人投篮高度（米）："))
distance = float(input("请输入投篮距离（米）："))

# 计算高度差
h = hoop_height - robot_height

# 计算投篮角度（弧度）
angle_rad = math.atan((h + math.sqrt(h**2 + distance**2)) / distance)

# 将弧度转换为角度
angle_deg = math.degrees(angle_rad)

# 计算出手速度
g = 9.81 # 重力加速度 (m/s^2)
v0 = math.sqrt((g * distance**2) / (2 * (math.cos(angle_rad))**2 * (distance * math.tan(angle_rad) - h)))

# 输出结果
print(f"计算出的投篮角度：{angle_deg:.2f} 度")
print(f"计算出的出手速度：{v0:.2f} m/s")
```

请输入篮圈高度（米）：3.05
请输入机器人投篮高度（米）：2
请输入投篮距离（米）：4.6
计算出的投篮角度：51.43 度
计算出的出手速度：7.52 m/s



案例2：投篮模拟



武汉大学
WUHAN UNIVERSITY

思考

- ✱ 通过轨迹模拟，篮球穿过篮圈了吗？
- ✱ 如何找到最优的投篮角度和出手速度组合？
- ✱ 如果需要考虑空气阻力，那么投篮模拟程序需要怎样修改呢？



案例2：投篮模拟



■ 拓展投篮模拟程序的功能：

✿ (4) 模拟投篮轨迹

- 使用抛物线运动公式计算轨迹，即每一时刻 t 篮球的 x 和 y 坐标值。

篮球坐标值 $x = v_x \cdot t$ ，坐标值 $y = v_y \cdot t - \frac{1}{2}gt^2$ ，其中 $v_x = v_0 \cdot \cos(\theta)$ 是水平速度，

$v_y = v_0 \cdot \sin(\theta)$ 是垂直速度。

✿ (5) 绘制轨迹

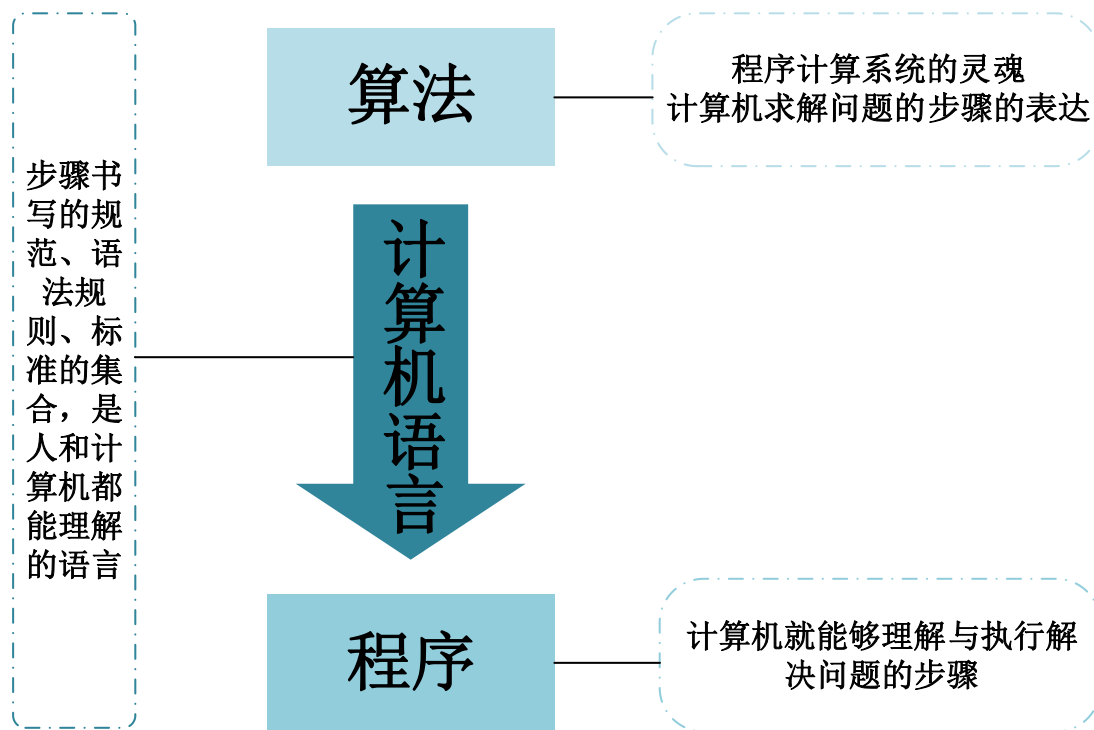
- 使用 `turtle` 模块绘制投篮轨迹，在图中标注篮圈位置和机器人位置。



小结



- 计算机只能解决计算问题，问题的计算部分由程序来完成，一般包括输入、处理和输出三部分（IPO）。
- 算法、计算机语言和程序之间的关系





小结



■ 怎样学习计算机编程

- ✱ 首先，掌握编程语言的语法，熟悉基本概念和逻辑。
- ✱ 其次，结合计算问题思考程序结构，会使用编程套路。
- ✱ 最后，参照案例多练习多实践，学会举一反三。

■ Python程序包含一系列语句，这些语句操作各种类型的数据。

■ 常用的基本数据类型包括整数、浮点数、复数、布尔值和字符串。

■ 数据都是对象，通过变量来引用。

■ 用运算符构造表达式完成各种运算。

■ 通过字符串格式化可以控制数据的输出格式。

■ 标准库的turtle模块支持绘图。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

